

Operational Methods in Impossibility Theory: A Research Program

Mohamed Amine Belachhab
Independent Researcher

October 2025

Abstract

We propose a research program based on operational approaches to impossibility results. The core idea is that operations which generate new mathematical objects can prove structural incompleteness across different domains. We identify five specific areas where this methodology might yield new results: type theory, proof assistants, complexity theory, model theory, and formal systems. For each area, we state precise research questions and explain why operational methods may succeed where classical techniques face limitations.

1 Introduction

1.1 The Core Principle

An operational impossibility proof has three components:

1. A collection S proposed to be “complete”
2. An operation A that maps elements to new elements
3. A proof that $A(S) \notin S$

When successful, this shows S cannot contain everything it claims to contain.

1.2 Why This Matters

Classical impossibility proofs (Russell, Cantor, Gödel) use indirect methods. Operational proofs are:

- **Constructive:** They build the counterexample explicitly
- **Algorithmic:** They can be implemented as computer programs
- **Portable:** They work in any domain with definable operations

1.3 This Paper’s Goal

We identify research directions where operational methods might prove useful. Each section asks: “Can we use operations to prove impossibility here?”

2 Type Theory

2.1 Background

Type theories use universe hierarchies: $\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$. This prevents paradoxes like “the type of all types.”

2.2 Research Question

Question 2.1. *Can we define a type-level operator T such that for any universe $\mathcal{U}$, the type $T(\mathcal{U})$ provably cannot be in $\mathcal{U}$?*

2.3 Why Operational Methods Help

Current proofs (Girard’s paradox) are complex. An operational proof would:

- Show explicitly why $T(\mathcal{U}) \notin \mathcal{U}$
- Extend naturally to exotic type systems
- Be mechanizable in proof assistants

2.4 Specific Problem

Definition 2.2. Define $T(\mathcal{U}) = (A : \mathcal{U}) \rightarrow \mathcal{U}$ (the type of type functions).

Conjecture 2.3. *If $T(\mathcal{U}) : \mathcal{U}$, then we can derive a contradiction using operational generation similar to the set-theoretic tagging argument.*

Remark 2.4. This would give an alternative proof of universe hierarchy necessity that emphasizes construction over impredicativity.

3 Proof Assistants

3.1 Background

Proof assistants (Lean, Coq, Agda) verify mathematical proofs mechanically. They manipulate formal objects: proofs, types, terms.

3.2 Research Question

Question 3.1. *Can operational methods be implemented as tactics that automatically find impossibility results?*

3.3 Why This Matters

If we can code up operational proofs, then:

- The proof assistant could find counterexamples automatically
- We could verify impossibility results mechanically
- The method becomes a reusable tool

3.4 Specific Problem

Proposition 3.2. *The following should be implementable in Lean 4:*

```
def operational_tag (S : Set ) : Set :=  
  {s | x S, s = tag x}
```

```
theorem impossibility_by_operation  
  (U : Set (Set ))  
  (h : S, S U) :  
  False := by  
  have : tag U U := rank_argument  
  have : tag U U := h (tag U)  
  contradiction
```

Remark 3.3. Mechanizing this would prove the method is rigorous enough for formal verification standards.

4 Complexity Theory

4.1 Background

Complexity classes (**P**, **NP**, **PSPACE**) are collections of languages. Diagonalization shows no complexity class can enumerate itself.

4.2 Research Question

Question 4.1. *Can we measure the “generative strength” of complexity class operations and classify them hierarchically?*

4.3 Why Operational Methods Help

Classical diagonalization is non-constructive. Operational methods could:

- Quantify how much operations exceed a given class
- Provide explicit witnesses for separations
- Connect to descriptive complexity

4.4 Specific Problem

Definition 4.2. For complexity class $\mathcal{C}$, define the diagonalization operation:

$$D(\mathcal{C}) = \{x : M_x \text{ is } \mathcal{C}\text{-machine and } x \notin L(M_x)\}$$

Conjecture 4.3. *The operation D has definable computational complexity $c(\mathcal{C})$, and classes can be ordered by this complexity. Moreover, $D(\mathcal{C}) \notin \mathcal{C}$ for all reasonable classes $\mathcal{C}$.*

Remark 4.4. This would give an operational interpretation of the halting problem and related undecidability results.

5 Model Theory

5.1 Background

Tarski's undefinability theorem: arithmetic truth is not definable in arithmetic. Similar to no universal set existing.

5.2 Research Question

Question 5.1. *Can semantic operations (forming theories, taking ultrapowers) be viewed as operations that prevent closure?*

5.3 Why This Matters

Tarski's proof uses diagonalization. An operational proof would:

- Show explicit construction of undefinable sets
- Extend to weaker logical systems
- Connect definability to computational generation

5.4 Specific Problem

Definition 5.2. For structure $\mathfrak{M}$, define:

$$\text{Th}(\mathfrak{M}) = \{\phi : \mathfrak{M} \models \phi\}$$

Conjecture 5.3. *The operation $\mathfrak{M} \mapsto \text{Th}(\mathfrak{M})$ generates objects (truth predicates) not expressible in $\mathfrak{M}$'s language. This can be proven operationally without diagonal lemma.*

Remark 5.4. Success would provide an alternative foundation for undefinability results based on generation rather than self-reference.

6 Unified Framework

6.1 The Meta-Question

Question 6.1. *Do Russell's Paradox, Gödel's Incompleteness, Turing's Halting Problem, and Tarski's Undefinability share a common operational structure?*

6.2 Proposed Framework

Definition 6.2 (Abstract Operational System). An operational system consists of:

- A collection of objects $\mathcal{O}$
- A family of operations $\mathcal{F} = \{f_i : \mathcal{O} \rightarrow \mathcal{O}\}$
- A closure predicate $\text{Closed}(S)$ meaning “ S is closed under all $f \in \mathcal{F}$ ”

Conjecture 6.3 (Unified Impossibility Principle). *If operations in $\mathcal{F}$ satisfy:*

1. **Injectivity:** $f(x) = f(y) \implies x = y$
2. **Growth:** *Some measure $\mu(f(x)) > \mu(x)$*
3. **Definability:** *Operations are expressible in the system*

Then no subset $S \subseteq \mathcal{O}$ can be closed under all operations while containing all objects up to some bound.

6.3 Why This Would Matter

A proof would:

- Unify disparate impossibility results
- Identify exactly what makes systems “incomplete”
- Apply to new domains not yet explored

6.4 Specific Problem

Theorem 6.4 (Meta-Theorem Sketch). *Let $(\mathcal{O}, \mathcal{F}, \mu)$ be an operational system where $\mu : \mathcal{O} \rightarrow \text{Ord}$ is a rank function and all $f \in \mathcal{F}$ increase rank. Then for any $S \subseteq \mathcal{O}$, there exists $f \in \mathcal{F}$ and $x \in S$ such that $f(x) \notin S$.*

Proof Sketch. Fix S with $\sup\{\mu(x) : x \in S\} = \alpha$. Choose $f \in \mathcal{F}$ with growth rate g . Then $\mu(f(S)) \geq \alpha + g > \alpha$, so $f(S)$ cannot be in S (all elements of S have rank $< \alpha + 1$). $\square$

Remark 6.5. This is deliberately abstract. Making it precise for each domain (sets, types, languages) is the research program.

7 Methodology

7.1 How to Pursue These Directions

For each research question:

1. **Formalize operations:** Define them precisely in the target domain
2. **Identify properties:** Prove injectivity, growth, definability
3. **Construct counterexamples:** Show operations generate objects outside proposed complete collections
4. **Mechanize when possible:** Implement in proof assistants

7.2 Success Criteria

A direction is successful if:

- It proves results not easily obtained classically
- It provides computational or constructive advantages
- It suggests generalizations to other areas

8 Limitations

8.1 What We Don't Claim

- We don't prove any of the stated conjectures here
- We don't guarantee operational methods will work in every domain
- We don't claim this approach replaces classical methods

8.2 What We Do Claim

- These are well-defined research questions
- Operational methods have advantages in specific contexts
- Success in even one area would be valuable

9 Conclusion

The operational approach to impossibility results suggests a general methodology applicable beyond set theory. We've identified five specific domains where this approach might yield new results:

1. Type theory universe hierarchies
2. Proof assistant automation
3. Complexity class separations
4. Model-theoretic undefinability
5. Unified impossibility framework

Each represents a concrete research direction with:

- Precise problem statements
- Clear motivation
- Potential for novel results

The fundamental open question is whether there exists a unified theory of closure-breaking operations that explains all impossibility results. Answering this would represent a genuine contribution to foundations.

Note: This document proposes a research program rather than presenting completed results. The conjectures stated are open problems that could constitute research projects.

References

- [1] Pierce, B. C. (2002). *Types and Programming Languages*. MIT Press.
- [2] Bertot, Y., & Castéran, P. (2004). *Interactive Theorem Proving and Program Development*. Springer.
- [3] Sipser, M. (2013). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.

- [4] Marker, D. (2002). *Model Theory: An Introduction*. Springer.
- [5] Smullyan, R. M. (1992). *Gödel's Incompleteness Theorems*. Oxford University Press.
- [6] Univalent Foundations Program. (2013). *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study.